

P2 功能综合方案

文档版本：v1.0

编写日期：2026 年 3 月 14 日

开发负责人：王舟

一、性能优化方案

1.1 缓存策略

Redis 缓存配置

```
// src/cache.rs
use redis::{Client, Connection, Commands};
use std::time::Duration;

pub struct CacheService {
    client: Client,
    default_ttl: Duration,
}

impl CacheService {
    pub fn new(redis_url: &str) -> Result<Self> {
        let client = Client::open(redis_url)?;
        Ok(Self {
            client,
            default_ttl: Duration::from_minutes(30),
        })
    }

    pub fn get_connection(&self) -> Result<Connection> {
        Ok(self.client.get_connection())
    }

    /// 缓存案件列表
    pub fn cache_case_list(
        &self,
        user_id: &str,
```

```

        cases: &Vec<Case>,
    ) -> Result<()> {
        let mut conn = self.get_connection()?;
        let key = format!("user: {}:cases", user_id);
        let ttl = Duration::from_minutes(5);

        conn.set_ex:::<_, _, ()>(&key, serde_json::to_string(cases)?, ttl.a
        Ok(())
    }

```

/// 获取缓存的案件列表

```

pub fn get_cached_case_list(&self, user_id: &str) -> Result<Option<Vec<Case>>> {
    let mut conn = self.get_connection()?;
    let key = format!("user: {}:cases", user_id);

    let cached: Option<String> = conn.get(&key)?;
    match cached {
        Some(data) => Ok(Some(serde_json::from_str(&data)?)),
        None => Ok(None),
    }
}

```

/// 缓存文件解析结果

```

pub fn cache_parsed_result(
    &self,
    evidence_id: &str,
    result: &ParsedResult,
) -> Result<()> {
    let mut conn = self.get_connection()?;
    let key = format!("evidence: {}:parsed", evidence_id);
    let ttl = Duration::from_hours(24);

    conn.set_ex:::<_, _, ()>(&key, serde_json::to_string(result)?, ttl.a
    Ok(())
}
}

```

内存缓存

```
// src/cache/memory.rs
use moka::future::Cache;
use std::time::Duration;

pub struct MemoryCache {
    cache: Cache<String, serde_json::Value>,
}

impl MemoryCache {
    pub fn new() -> Self {
        Self {
            cache: Cache::builder()
                .max_capacity(10_000)
                .time_to_live(Duration::from_minutes(30))
                .time_to_idle(Duration::from_minutes(10))
                .build(),
        }
    }

    pub async fn get(&self, key: &str) -> Option<serde_json::Value> {
        self.cache.get(key).await
    }

    pub async fn set(&self, key: String, value: serde_json::Value) {
        self.cache.insert(key, value).await;
    }

    pub async fn invalidate(&self, key: &str) {
        self.cache.invalidate(key).await;
    }
}
```

1.2 数据库优化

查询优化

```
// 使用 EXPLAIN 分析查询
sqlx::query("EXPLAIN QUERY PLAN SELECT * FROM event_nodes WHERE case_id =
```

```

        .bind(&case_id)
        .fetch_all(&pool)
        .await?;

// 优化后：添加索引
CREATE INDEX idx_event_nodes_case_time ON event_nodes(case_id, event_time)

// 批量查询代替 N+1 查询
// ❌ 错误：N+1 查询
for case in cases {
    let nodes = get_nodes(&case.id).await?;
}

// ✅ 正确：批量查询
let case_ids: Vec<_> = cases.iter().map(|c| &c.id).collect();
let nodes = get_nodes_batch(&case_ids).await?;

```

连接池优化

```

// SQLx 连接池配置
use sqlx::sqlite::SqlitePoolOptions;

let pool = SqlitePoolOptions::new()
    .max_connections(20)           // 最大连接数
    .min_connections(5)           // 最小连接数
    .acquire_timeout(Duration::from_secs(30))
    .idle_timeout(Duration::from_secs(600))
    .connect(&database_url)
    .await?;

```

1.3 CDN 加速

```

# Nginx CDN 配置
location /static/ {
    alias /opt/legal-minds/static;

    # 浏览器缓存
    expires 30d;
    add_header Cache-Control "public, immutable";

    # 压缩

```

```

    gzip on;
    gzip_types text/css application/javascript image/svg+xml;
}

location /api/v1/files/*/download {
    # 大文件使用 sendfile
    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;

    # 限制速率
    limit_rate 10m;
}

```

1.4 性能指标

指标	当前	目标	优化手段
API P95	500ms	200ms	缓存 + 索引
搜索响应	300ms	100ms	FTS5 优化
文件上传	2s	1s	分块上传
页面加载	2s	1s	CDN + 压缩

二、多端适配方案

2.1 响应式布局

```

// frontend/src/utils/responsive.rs

#[derive(Clone, Copy, PartialEq)]
pub enum Breakpoint {
    Sm,    // 640px
    Md,    // 768px
    Lg,    // 1024px
    Xl,    // 1280px
}

pub fn use_breakpoint() -> Signal<Breakpoint> {
    let mut breakpoint = use_signal(|| Breakpoint::Lg);

```

```

use_effect(move || {
    let update_breakpoint = move || {
        let width = window().inner_width().unwrap() as f64;
        let new_breakpoint = if width < 640.0 {
            Breakpoint::Sm
        } else if width < 768.0 {
            Breakpoint::Md
        } else if width < 1024.0 {
            Breakpoint::Lg
        } else {
            Breakpoint::Xl
        };

        if breakpoint() != new_breakpoint {
            breakpoint.set(new_breakpoint);
        }
    };

    update_breakpoint();

    let closure = Closure::wrap(Box::new(move |_| {
        update_breakpoint();
    }) as Box<dyn FnMut(>));

    window().add_event_listener_with_callback("resize", &closure).unwrap();
    closure.forget();
});

breakpoint
}

// 使用示例
rsx! {
    if use_breakpoint() == Breakpoint::Sm {
        // 移动端布局
        MobileLayout {}
    } else {
        // 桌面端布局
        DesktopLayout {}
    }
}

```

```
    }  
  }  
}
```

2.2 移动端优化

```
// 底部导航栏（移动端）  
#[component]  
pub fn BottomNav() -> Element {  
  rsx! {  
    nav { class: "bottom-nav",  
      Link { to: "/cases",  
        icon::Folder {}  
        span { "案件" }  
      }  
      Link { to: "/timeline",  
        icon::Timeline {}  
        span { "时间轴" }  
      }  
      Link { to: "/persons",  
        icon::Person {}  
        span { "人物" }  
      }  
      Link { to: "/search",  
        icon::Search {}  
        span { "搜索" }  
      }  
      Link { to: "/profile",  
        icon::AccountCircle {}  
        span { "我的" }  
      }  
    }  
  }  
}
```

2.3 PWA 支持

```
// manifest.json  
{  
  "name": "法律认知引擎",  
  "short_name": "法律引擎",  
  "start_url": "/",  
}
```

```
"display": "standalone",
"background_color": "#ffffff",
"theme_color": "#1a365d",
"icons": [
    {
        "src": "/icon-192.png",
        "sizes": "192x192",
        "type": "image/png"
    },
    {
        "src": "/icon-512.png",
        "sizes": "512x512",
        "type": "image/png"
    }
]
}
```

三、数据迁移方案

3.1 SQLite → PostgreSQL

迁移脚本

```
#!/usr/bin/env python3
# migrate_to_postgres.py

import sqlite3
import psycopg2
from psycopg2.extras import execute_batch

# 连接 SQLite
sqlite_conn = sqlite3.connect('legal_minds.db')
sqlite_conn.row_factory = sqlite3.Row

# 连接 PostgreSQL
pg_conn = psycopg2.connect(
    host='localhost',
    database='legal_minds',
    user='legal_minds',
```

```

        password='password'
    )

def migrate_table(table_name, columns):
    """迁移单个表"""
    # 从 SQLite 读取
    sqlite_cursor = sqlite_conn.cursor()
    sqlite_cursor.execute(f"SELECT {'', '.join(columns)} FROM {table_name}")
    rows = sqlite_cursor.fetchall()

    # 写入 PostgreSQL
    pg_cursor = pg_conn.cursor()
    placeholders = ', '.join(['%s'] * len(columns))
    insert_sql = f"""
        INSERT INTO {table_name} ({', '.join(columns)})
        VALUES ({placeholders})
        ON CONFLICT DO NOTHING
    """

    execute_batch(pg_cursor, insert_sql, rows, page_size=1000)
    pg_conn.commit()

    print(f"迁移完成: {table_name} ({len(rows)} 条)")

# 迁移所有表
tables = {
    'users': ['id', 'username', 'email', 'password_hash', 'created_at'],
    'cases': ['id', 'name', 'description', 'owner_id', 'created_at'],
    'evidence_files': ['id', 'case_id', 'original_name', 'parsed_text', 'created_at'],
    'event_nodes': ['id', 'case_id', 'title', 'event_time', 'created_at'],
    # ...
}

for table, columns in tables.items():
    migrate_table(table, columns)

sqlite_conn.close()
pg_conn.close()

print("=== 迁移完成 ===")

```

3.2 数据校验

```
# verify_migration.py

import sqlite3
import psycopg2

def count_rows(conn, table):
    cursor = conn.cursor()
    cursor.execute(f"SELECT COUNT(*) FROM {table}")
    return cursor.fetchone()[0]

# 对比数据量
sqlite_conn = sqlite3.connect('legal_minds.db')
pg_conn = psycopg2.connect(...)

tables = ['users', 'cases', 'evidence_files', 'event_nodes']

print("数据量对比:")
for table in tables:
    sqlite_count = count_rows(sqlite_conn, table)
    pg_count = count_rows(pg_conn, table)

    if sqlite_count == pg_count:
        print(f"✅ {table}: {sqlite_count} = {pg_count}")
    else:
        print(f"❌ {table}: {sqlite_count} ≠ {pg_count}")

sqlite_conn.close()
pg_conn.close()
```

3.3 回滚方案

```
#!/bin/bash
# rollback_migration.sh

echo "=== 开始回滚 ==="

# 1. 停止服务
systemctl stop legal-minds
```

```
# 2. 备份 PostgreSQL 数据
pg_dump legal_minds > backup_pg_$(date +%Y%m%d).sql

# 3. 恢复 SQLite
cp legal_minds.db.backup legal_minds.db

# 4. 切换数据库配置
sed -i 's/postgres/sqlite/' /opt/legal-minds/.env

# 5. 启动服务
systemctl start legal-minds

echo "=== 回滚完成 ==="
```

四、版本更新规范

4.1 语义化版本

版本号格式：MAJOR.MINOR.PATCH

MAJOR：不兼容的 API 变更
MINOR：向后兼容的功能新增
PATCH：向后兼容的问题修复

示例：

v1.0.0 - 首期发布
v1.1.0 - 新增搜索功能
v1.1.1 - 修复搜索 Bug
v2.0.0 - 二期发布 (AI 功能)

4.2 更新日志模板

更新日志

[1.1.0] - 2026-03-21

新增

- 全文搜索功能
- 搜索建议

- 搜索历史

优化

- 搜索性能提升 50%
- 搜索结果相关性优化

修复

- 修复文件上传超时问题
- 修复时间轴显示异常

[1.0.1] - 2026-03-14

修复

- 修复登录 Token 过期问题
- 修复案件列表分页错误

4.3 发布流程

```
#!/bin/bash
```

```
# release.sh
```

```
VERSION=$1
```

```
if [ -z "${VERSION}" ]; then
    echo "用法: release.sh <版本号>"
    exit 1
fi
```

```
# 1. 更新版本号
```

```
sed -i "s/version = \".*\"/version = \"${VERSION}\"/" Cargo.toml
```

```
# 2. 更新更新日志
```

```
vim CHANGELOG.md
```

```
# 3. 提交
```

```
git add -A
```

```
git commit -m "release: v${VERSION}"
```

```
git tag -a "v${VERSION}" -m "Release v${VERSION}"
```

```
# 4. 推送
```

```
git push origin main
```

```
git push origin --tags
```

```
# 5. 构建发布
cargo build --release
```

```
# 6. 创建 GitHub Release
gh release create "v${VERSION}" \
  --title "Release v${VERSION}" \
  --notes-file CHANGELOG.md \
  target/release/legal-minds-server
```

4.4 兼容性策略

版本	兼容性	说明
v1.x → v1.y	完全兼容	小版本升级
v1.x → v2.0	需要迁移	大版本升级
数据库	向后兼容	自动迁移脚本

五、总结

5.1 P2 功能清单

功能	工作量	优先级
性能优化	20 人天	P2
多端适配	15 人天	P2
数据迁移	10 人天	P2
版本规范	5 人天	P2
合计	50 人天	-

5.2 实施顺序

- 性能优化（上线后立即进行）
- 多端适配（用户需求驱动）
- 数据迁移（PostgreSQL 需求时）
- 版本规范（持续执行）

开发负责人：王舟

邮箱：main@mails.wedevs.org

电话：15378391447